

TaskFlow

Mon oral, étape par étape

Version adaptée aux consignes du professeur - 7 octobre 2026

Le cadre de la présentation

Le livret étudiant V2 prévoit une **présentation individuelle de dix minutes suivie de questions**. Les horaires et la durée des questions sont à confirmer par l'établissement. Le fichier du professeur précise l'organisation des pages et des composants React ; il ne modifie pas cette durée.

Ce support présente **TaskFlow, sujet A**, une application de tâches personnelles. Le discours explique le besoin, les changements effectués, l'utilisation et les vérifications. Il évite une lecture détaillée du code.

Comment s'en servir

Les paragraphes du discours peuvent être dits à voix haute, avec ses propres mots. Les encadrés textuels à montrer servent de repères pour les manipulations et ne sont pas à lire. Le temps comprend la parole, les clics et les pauses. Répéter avec un chronomètre reste nécessaire : ce document ne garantit pas une durée exacte au mot près.

Temps	Partie
0 :00–1 :00	Présenter le sujet et son utilité.
1 :00–2 :30	Expliquer la prise en compte des consignes.
2 :30–5 :30	Montrer le parcours de l'utilisateur.
5 :30–7 :00	Expliquer simplement les données et leur protection.
7 :00–8 :30	Présenter les tests et les documents.
8 :30–10 :00	Faire le bilan et évoquer les améliorations.

Documents et version

Dépôt : <https://github.com/shesaidimnothing/taskflow-fullstack-js>

Version visée par ce support : **v1.1.0**, adaptation aux consignes du professeur. Le tag **v1.0.0** conserve la livraison initiale. Le dépôt est privé et l'application se démontre en local.

Conseil : Ouvrir l'application, Swagger et le résultat des tests avant de commencer. Les noms techniques servent de repères, pas de liste à réciter. Les questions en fin de document sont hors des dix minutes.

1. Le sujet et le besoin

Temps : 0 :00 à 1 :00

Bonjour, je vais vous présenter TaskFlow. C'est une application qui permet de gérer ses tâches personnelles. Parmi les trois sujets proposés dans le livret, le projet correspond au sujet A. L'objectif est de retrouver au même endroit ce qu'on doit faire et de suivre son avancement.

Par exemple, pour préparer une présentation, on peut créer une tâche, ajouter quelques indications et prévoir une échéance. Ensuite, on fait évoluer son statut de *à faire* à *en cours*, puis à *terminée*. Chaque utilisateur possède son propre compte et sa propre liste.

La présentation va suivre le parcours d'une personne qui utilise l'application. Je vais d'abord expliquer comment les consignes du professeur ont été prises en compte, puis faire une démonstration. Je terminerai par les vérifications réalisées et les limites de cette version.

À montrer : Afficher la page de connexion. Annoncer le sujet A et le besoin utilisateur.

2. Les consignes et les changements

Temps : 1 :00 à 2 :30

Les consignes du professeur demandent d'abord deux pages distinctes : une pour se connecter et une pour créer un compte. Chacune doit contenir un titre et un formulaire avec un email et un mot de passe. Dans la première version, ces deux fonctions étaient regroupées dans un même composant. Elles sont maintenant séparées dans LoginPage et Registerpage.

Autre point demandé : récupérer ce que l'utilisateur écrit avec `useState`. Concrètement, la valeur de chaque champ est suivie par React au fur et à mesure de la saisie, puis envoyée au serveur quand le formulaire est validé. Cela rend le fonctionnement des formulaires plus clair par rapport à l'exercice.

La navigation a aussi été revue. Une barre de navigation permet de passer de la connexion à l'inscription. React Router gère les pages et leurs adresses. On peut ouvrir directement une page ou utiliser le bouton Retour du navigateur. Si on n'est pas connecté, l'espace des tâches renvoie vers la connexion.

Enfin, l'interface métier est découpée en trois composants : TaskList affiche la liste, TaskItem affiche une tâche et TaskForm sert à la créer ou la modifier. L'objectif n'est pas d'ajouter des écrans inutiles : c'est de donner un rôle précis à chaque partie, comme demandé dans les consignes.

À montrer : Passer de Connexion à Inscription avec la navbar. Si nécessaire, montrer brièvement les noms des composants, sans ouvrir leur code ligne par ligne.

3. Utiliser TaskFlow

Temps : 2 :30 à 5 :30

Je vais maintenant montrer le parcours complet. Je commence par créer un compte de démonstration. Je renseigne une adresse email et un mot de passe. Une fois le compte créé, j'arrive dans mon espace. La liste est vide puisque je n'ai encore ajouté aucune tâche.

Je crée une première tâche intitulée "Préparer la présentation". Je peux préciser ce que je veux faire dans la description et ajouter une échéance. Je laisse le statut sur "à faire", puis j'enregistre. La tâche apparaît dans la liste avec les informations utiles, ce qui permet de comprendre rapidement où j'en suis.

En cliquant dessus, j'ouvre le détail. Je peux relire sa description, puis la modifier. Ici, je vais la passer en "en cours". Après l'enregistrement, le nouveau statut apparaît. Je recharge aussi la page : la tâche est toujours là, avec la modification qui vient d'être faite.

Pour la suppression, l'application demande une confirmation. Je peux annuler si je me suis trompé. Si je confirme, la tâche disparaît de ma liste. Cela couvre les actions principales demandées : créer, consulter, modifier et supprimer.

Le menu utilisateur permet aussi d'accéder à Mon compte pour consulter son profil et changer son mot de passe. Cette fonction déjà ajoutée au projet a été conservée. Je termine en montrant le compte B, préparé dans un autre navigateur. Sa liste ne contient pas les tâches du compte A. Chaque personne retrouve donc son propre espace. Je vais maintenant expliquer simplement comment l'application conserve ces informations et contrôle les accès.

À montrer : Prévoir environ deux minutes de manipulations : inscription ; création ; détail ; modification ; rechargement ; suppression annulée puis confirmée ; affichage du compte B. Garder une seconde tâche de A pour illustrer la séparation des listes.

4. Les données et la protection des comptes

Temps : 5 :30 à 7 :00

L'application repose sur trois parties. React correspond à ce qu'on voit dans le navigateur. Express reçoit les demandes et applique les règles. MongoDB conserve les comptes et les tâches. Quand on enregistre une tâche, l'interface envoie une demande au serveur avec fetch, puis affiche la réponse obtenue.

L'intérêt de la base de données, c'est que les tâches ne sont pas seulement présentes dans la page ouverte. Elles restent enregistrées quand on recharge le navigateur ou quand l'API redémarre. Un test spécifique vérifie d'ailleurs ce deuxième cas.

Pour les comptes, le mot de passe n'est pas conservé en clair. Le serveur stocke une empreinte calculée avec bcrypt. Après une connexion réussie, il remet un jeton qui permet de reconnaître l'utilisateur pendant ses prochaines demandes.

Mais être connecté ne donne pas accès à toutes les tâches. Le serveur vérifie aussi à qui appartient chaque donnée. Le contrôle se fait donc côté serveur, même si quelqu'un contourne les boutons de l'interface. Les erreurs de saisie sont également vérifiées : par exemple, un titre vide ou une date impossible sont refusés.

À montrer : Rester sur l'application. Un schéma oral suffit : navigateur, serveur, base de données. La cryptographie et les requêtes MongoDB peuvent être réservées aux questions.

5. Les vérifications et la documentation

Temps : 7 :00 à 8 :30

Pour vérifier que l'application ne fonctionne pas seulement dans un cas idéal, plusieurs contrôles sont prévus. Les tests API vérifient les opérations sur les tâches, les données invalides et les problèmes de connexion. Ils vérifient aussi que le compte B ne peut pas lire, modifier ou supprimer une tâche appartenant à A.

La suite API compte 37 tests. Quatre tests de parcours dans un navigateur complètent ces contrôles. Ils vérifient notamment les formulaires, les modifications de tâches et l'affichage sur mobile. Le nouveau test ajouté pour les consignes vérifie la navigation, les liens directs, le retour en arrière et la redirection quand la session n'est plus valide.

Les tests utilisent des données fictives dans des bases temporaires. Ils ne suppriment pas les tâches de développement. Le projet contient aussi un contrôle de qualité du code et une commande de construction de l'interface.

Côté documentation, le README donne les étapes pour installer et lancer le projet. Swagger permet de retrouver les endpoints, de voir les champs attendus et d'essayer les requêtes. Des exemples ont été ajoutés pour les demandes et les réponses, comme demandé par le professeur. Le dépôt contient également une comparaison entre les consignes et les changements réalisés.

À montrer : Montrer le résultat des tests, puis un endpoint Swagger avec son exemple. Consulter GitHub Actions pour le statut du commit présenté ; ne pas relancer toute l'installation pendant l'oral.

6. Le bilan et les améliorations possibles

Temps : 8 :30 à 10 :00

Le résultat répond au besoin de départ : un utilisateur peut se créer un compte, se connecter et gérer ses tâches dans un espace personnel. La nouvelle version reprend aussi l'organisation demandée par le professeur, avec des pages de connexion et d'inscription séparées, une vraie navigation et des composants dédiés aux tâches.

L'intérêt de ces changements est aussi de rendre le projet plus facile à comprendre. Si on souhaite modifier la présentation d'une tâche, on sait où intervenir. Si on veut changer le formulaire, on retrouve la partie correspondante sans chercher dans tout l'écran.

Il reste néanmoins des limites. Par exemple, la récupération d'un mot de passe oublié et la limitation des tentatives de connexion ne sont pas prévues. Pour une utilisation plus importante, il faudrait également mieux gérer les longues listes et approfondir les vérifications sur différents navigateurs.

Les bonus du livret, comme les priorités ou les statistiques, n'ont pas été ajoutés dans cette version. Les consignes les présentent comme des objectifs supplémentaires. La priorité a été de garder un parcours complet et de vérifier les fonctionnalités obligatoires avant d'élargir le projet.

Pour finir, TaskFlow montre comment une interface, un serveur et une base de données peuvent fonctionner ensemble pour répondre à un besoin simple. La démonstration permet de voir les fonctionnalités, et les tests apportent des vérifications complémentaires. Merci pour votre attention, je suis prêt à répondre à vos questions.

À montrer : Revenir à la liste des tâches. Terminer sur le résultat et les prochaines améliorations, sans lancer une nouvelle démonstration.

Avant l'oral : préparation rapide

1. Démarrer MongoDB puis l'application avant le passage. Tester la connexion et garder les terminaux ouverts.
2. Préparer deux comptes fictifs, A et B, dans deux contextes de navigation distincts. Conserver une tâche de A et une liste B vide.
3. Ouvrir les onglets utiles : application, Swagger, tests et dépôt GitHub.
4. Ne pas afficher de mot de passe personnel ni le contenu du fichier de configuration contenant la clé JWT.
5. Répéter avec les clics : si la présentation dépasse dix minutes, raccourcir les exemples et garder le parcours utilisateur complet.

Depuis la racine du projet, terminal 1 :

```
npm run db:local
```

Terminal 2 :

```
npm run dev
```

Application : `http://localhost:5173`

Swagger : `http://localhost:3000/api/docs/`

Contrôles à préparer avant le passage :

```
npm run check  
npm run test:persistence  
npm run test:e2e
```

Pour Playwright, installer Chromium au préalable avec `npx playwright install chromium`, ou utiliser Chrome installé selon les indications du README. Ne pas lancer un deuxième service sur un port déjà occupé.

Les changements à retenir en une minute

- Avant : connexion et inscription réunies. Après : deux pages distinctes.
- Avant : valeurs récupérées à l'envoi du formulaire. Après : champs suivis avec `useState`.
- Avant : changement d'écran interne. Après : navbar, routes et liens avec React Router.
- Avant : interface métier regroupée. Après : `TaskList`, `TaskItem` et `TaskForm`.
- En complément : exemples Swagger et test de navigation ; le CRUD et la protection des comptes sont conservés.

Questions possibles : réponses courtes

Ces réponses sont à préparer pour l'échange après la présentation, pas à réciter pendant les dix minutes.

Pourquoi TaskFlow ?

C'est le sujet A du catalogue, consacré aux tâches personnelles. Le dépôt de départ contenait déjà une ébauche autour de cette donnée métier. Un seul sujet a été réalisé.

À quoi sert useState dans les formulaires ?

Il permet à React de garder la valeur des champs pendant la saisie. À chaque modification, l'état est mis à jour ; à l'envoi, les valeurs courantes sont transmises au serveur.

Pourquoi React Router ?

Il associe une adresse à une page et permet de naviguer avec des liens. Ici, les pages principales sont /login, /register, /tasks et /account. Link et NavLink gèrent les liens ; Navigate sert aux redirections.

Pourquoi séparer TaskList, TaskItem et TaskForm ?

La liste organise les tâches, un item affiche une tâche et le formulaire gère la saisie. Chaque composant a un rôle simple, ce qui facilite la lecture et les changements.

Comment le front communique-t-il avec le serveur ?

Un client centralisé utilise fetch. Il envoie les données en JSON et ajoute le jeton pour les routes protégées. Le navigateur affiche ensuite le résultat ou un message d'erreur.

Pourquoi B ne voit-il pas les tâches de A ?

Le serveur filtre les données selon le compte connecté et vérifie le propriétaire pour chaque objet. Masquer un bouton ne suffirait pas, puisque l'API peut être appelée directement.

Quelle différence entre 401 et 404 ?

401 correspond à une authentification absente ou refusée. 404 indique qu'une tâche n'est pas accessible au compte : elle est absente ou appartient à quelqu'un d'autre.

Que se passe-t-il au redémarrage ?

Les données sont dans MongoDB. Redémarrer Express ne les efface pas. Le projet contient un test qui le vérifie réellement.

Quelle limite présente la session ?

Le jeton est dans sessionStorage et reste accessible au JavaScript. Une faille XSS pourrait le lire. La déconnexion efface la copie locale mais ne révoque pas un jeton déjà copié, qui expire après une heure.

Quelle aide a été utilisée ?

Le projet a été préparé avec Codex pour le code, les tests et la documentation. Présenter honnêtement les vérifications et la compréhension personnelle, selon les règles effectivement communiquées par l'établissement.